

Lista de Exercícios

Coleta de Dados na Web — do Scraper ao RAG

Hora de prática acompanhada · use com o notebook `web_scraper_didatico.ipynb`

Como usar. Os exercícios estão em ordem de dificuldade crescente. Os marcados como **[Guiado]** são para começar; **[Intermediário]** exigem modificar o código; **[Extensão]** são desafios abertos.

Antes de começar

- Abra o notebook no Colab e execute as células dos Blocos 1 a 4 (definem as funções).
- Para testar sem a planilha, use a semente manual sugerida no Bloco 5.
- Mantenha `MAX_PAGINAS` baixo enquanto experimenta.

Exercício 1 — Ajustando a profundidade **[Guiado]**

Objetivo. Entender, na prática, o efeito do parâmetro `profundidade_max` sobre o volume coletado.

Tarefa. Usando uma única semente, rode a coleta três vezes, com `profundidade_max` igual a 0, 1 e 2. Anote o número de páginas coletadas em cada caso e os níveis presentes na saída.

```
PAGINAS_SEMTE = ['https://www.iana.org/help/example-domains']

for prof in [0, 1, 2]:
    r = coletar(PAGINAS_SEMTE, profundidade_max=prof,
               max_paginas=40, delay=0.3)
    niveis = sorted({x['profundidade'] for x in r})
    print(f'profundidade {prof}: {len(r)} páginas, níveis {niveis}')
```

Pergunta. Por que o número de páginas tende a crescer rapidamente a cada nível? O que isso implica para um site grande?

Exercício 2 — Lendo o robots.txt **[Guiado]**

Objetivo. Verificar como diferentes sites declaram suas permissões para robôs.

Tarefa. Use a função `pode_acessar` para testar três URLs de domínios diferentes (escolha sites de organismos do corpus, como ISO, OMS, NIST). Observe o que cada um retorna — políticas de robots.txt mudam com o tempo, então confirme o resultado no dia da aula em vez de supor qual site bloqueia.

```
testes = [
    'https://www.iso.org/standard/81230.html',
    'https://www.who.int/',
    'https://www.iana.org/help/example-domains',
]
for u in testes:
    print(('permitido' if pode_acessar(u) else 'BLOQUEADO'), u)
```

Pergunta. Se um site bloqueia uma página no robots.txt, qual deve ser o comportamento do scraper? E se ele retorna erro 403 ao baixar?

Exercício 3 — Interpretando o conteúdo coletado [Intermediário]

Objetivo. Avaliar criticamente a qualidade do que foi extraído.

Tarefa. Após uma coleta, inspecione o primeiro registro. Verifique se o **texto** contém ruído de menu/rodapé (palavras repetidas, itens de navegação). Conte links e figuras.

```
r = resultados[0]
print('Título:', r['titulo'])
print('Nº links:', len(r['links']))
print('Nº figuras:', len(r['figuras']))
print('Trecho do texto:')
print(r['texto'][:400])
```

Pergunta. O texto parece limpo (só o conteúdo central) ou há resíduos de navegação? Como isso afetaria um RAG construído sobre este corpus? E ao inspecionar os registros das normas da ISO acessadas pelo visualizador OBP (URLs iso.org/obp/ui/...#iso:std:...), o que você observa no campo texto — e por quê?

Exercício 4 — Acrescentando um teto por domínio [Intermediário]

Objetivo. Compreender as salvaguardas que impedem um único site de dominar a coleta.

Tarefa. Com duas ou mais sementes de domínios diferentes, rode a coleta com **max_por_dominio=5** e depois com **max_por_dominio=20**. Use o **Counter** abaixo para contar quantas páginas vieram de cada domínio.

```
from collections import Counter
from urllib.parse import urlparse

r = coletar(PAGINAS_SEMENTE, profundidade_max=2,
            max_paginas=80, max_por_dominio=5, delay=0.3)

por_dom = Counter(urlparse(x['url']).netloc for x in r)
for dom, n in por_dom.items():
    print(f'{n:3} {dom}')
```

Pergunta. Por que um teto por domínio é útil quando coletamos de muitos sites ao mesmo tempo? O que aconteceria sem ele?

Exercício 5 — Caçando PDFs disfarçados [Extensão]

Objetivo. Medir o impacto da verificação real de tipo (HEAD) na filtragem de PDFs.

Tarefa. Leia as sementes da planilha duas vezes: com `verificar_tipo_real=True` e com `False`. Compare quantos PDFs são descartados em cada caso e liste os que escapam sem a verificação.

```
com = ler_sementes_da_planilha(URL_PLANILHA, verificar_tipo_real=True)
sem = ler_sementes_da_planilha(URL_PLANILHA, verificar_tipo_real=False)

# Os que 'sem' tratou como semente mas 'com' descartou = PDFs disfarçados
disfarcados = set(sem) - set(com)
print(f'PDFs que escapam sem a verificação real: {len(disfarcados)}')
for u in disfarcados:
    print(' -', u)
```

Pergunta. Que padrões de URL aparecem entre os disfarçados? Por que a simples checagem da extensão `.pdf` não basta?

Exercício 6 — Do corpus ao chunk [Extensão]

Objetivo. Conectar a coleta à primeira etapa de um pipeline de RAG.

Tarefa. Use a função `chunk_simples` do Bloco 6 sobre o corpus coletado. Varie o `tamanho` do chunk (ex.: 300, 800, 1500) e observe quantos trechos cada página gera. Para cada chunk, mantenha a `url` de origem associada (proveniência).

```
for tam in [300, 800, 1500]:
    total = sum(len(chunk_simples(r['texto'], tamanho=tam))
                for r in resultados)
    print(f'tamanho {tam}: {total} chunks no corpus todo')

# Cada chunk deve carregar sua fonte para o RAG poder citar:
exemplo = resultados[0]
chunks = [{'fonte': exemplo['url'], 'texto': c}
           for c in chunk_simples(exemplo['texto'], tamanho=800)]
print(chunks[0]['fonte'])
```

Pergunta. Chunks muito grandes ou muito pequenos — como cada extremo afeta a recuperação e a geração de respostas no RAG? Por que associar a fonte a cada chunk é essencial?